PCNT del ESP32

y encoders en cuadratura

Guía práctica con Arduino IDE (core Arduino-ESP32 3.x) — teoría, hardware, código y diagnóstico

Contexto	Rover ERC · Subsistema Electrónica y Potencia · modelo a escala (Ackermann 4WD)
Motor / encoder	CQRobot CQR37D12V64EN-I: 90:1, 120 rpm de salida, 64 CPR en el eje del motor, encoder alimentado a 3,3 V
Conectores	J9 (FR), J16 (FL), J18 (RL), J8 (RR) — Molex KK-254, 4 vías
Fuente	Basada en la nota «Encoders en cuadratura y periférico PCNT» (vigente, 2026-09-16) y en la documentación oficial del driver PCNT de ESP-IDF
Archivos	Cada ejemplo de código de esta guía se entrega además como sketch .ino listo para abrir

Contenido

1. La idea en una página
2. El encoder en cuadratura
3. Cómo es el PCNT por dentro
4. Decodificación x4 con dos canales
5. El filtro de glitch
6. El desborde del contador de 16 bits
7. El driver en Arduino IDE
8. Ejemplos de código (0 a 5)
9. De cuentas a velocidad
10. Hardware: qué hace la placa por el firmware
11. Diagnóstico de problemas
12. Checklist de banco para el rover
13. Apéndices: driver viejo, referencia rápida de la API, números del rover

1. La idea en una página

El **PCNT** (*Pulse Counter*) es un periférico de hardware del ESP32 que **cuenta flancos de señales digitales sin intervención de la CPU**. Le conectás una o dos señales, le decís qué hacer en cada flanco (sumar, restar o nada) y él mantiene un contador de 16 bits que tu programa lee cuando quiere. Entre lectura y lectura, el procesador está libre.

Una analogía: contar vueltas de una rueda con interrupciones es como tener a una persona que, cada vez que la rueda pasa por una marca, deja lo que está haciendo y anota un palito en un cuaderno. El PCNT es un **odómetro mecánico** acoplado directamente a la rueda: cuenta solo, y vos lo mirás cuando necesitás el dato.

¿Por qué importa en el rover?

Hagamos la cuenta de cuántos flancos hay que procesar con los cuatro motores a velocidad máxima:

Magnitud	Cálculo	Valor
Velocidad del eje del motor	120 rpm de salida × 90	10 800 rpm = 180 rev/s
Pulsos por canal	180 rev/s × 16 pulsos/rev	2880 Hz por canal
Flancos por rueda (A y B, subida y bajada)	2880 × 4	11 520 flancos/s
Flancos totales (4 ruedas)	11 520 × 4	46 080 flancos/s
Con interrupciones, a ~3 µs por ISR	46 080 × 3 µs	≈ 0,14 s de CPU por segundo (≈14 % de un núcleo)
Con PCNT	—	0 interrupciones por flanco

Además del consumo de CPU, las interrupciones tienen dos problemas más sutiles: **jitter** (otras interrupciones de WiFi, del sistema o de escritura en flash las demoran) y **pérdida de flancos** si dos llegan más rápido de lo que la ISR los procesa. Un flanco perdido en cuadratura es un error de posición permanente. El PCNT sincroniza y cuenta cada flanco en hardware, a la frecuencia del reloj APB (80 MHz): no se pierde nada a 2,9 kHz.

> **Clave.** Lo que tenés que llevarte de esta guía: (1) el ESP32 tiene **8 unidades PCNT**, cada una con **2 canales**; (2) con los dos canales cruzados (A↔B) una unidad decodifica un encoder en **x4** con dirección; (3) el contador es de **16 bits** y en el rover **vuelve a 0 cada 2,8 s**, así que hay que acumularlo; (4) el **filtro de glitch** es gratis y elimina ruido sin componentes; (5) en Arduino IDE con el core 3.x se usa el driver `driver/pulse_cnt.h`.

2. El encoder en cuadratura

El encoder del CQRobot es magnético (efecto Hall): un disco con imanes gira solidario al eje del motor y dos sensores, desplazados mecánicamente, generan dos ondas cuadradas llamadas **A** y **B**. Están desfasadas **90° eléctricos** (un cuarto de período). Ese desfase es lo que permite saber el **sentido de giro**: si A cambia antes que B, el eje gira en un sentido; si B cambia antes que A, gira en el otro.

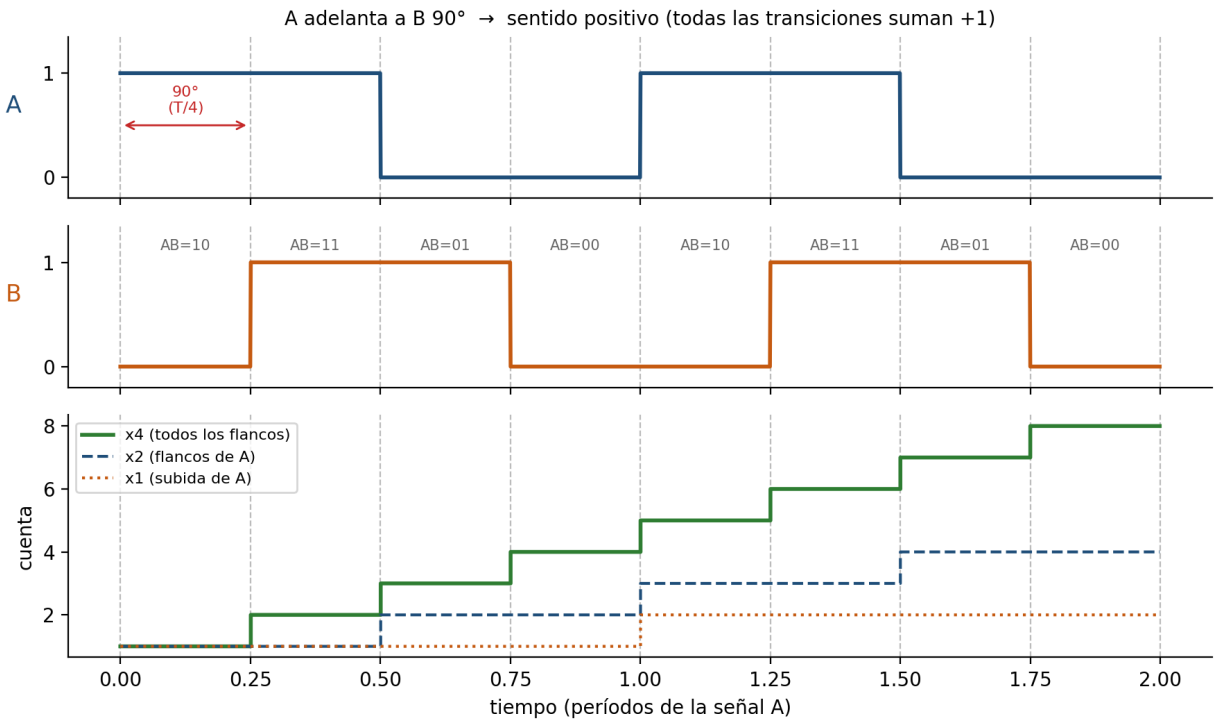


Figura 1 — Señales A y B en cuadratura. Abajo: cuánto cuenta cada modo de decodificación en el mismo intervalo. En x4 cada uno de los 4 flancos por período suma una cuenta.

Por qué el desfase de 90° da la dirección

Mirá los valores de AB en la Figura 1: la secuencia es 10 → 11 → 01 → 00 → 10… En cada paso **cambia un solo bit** (es un código Gray). Si el eje gira al revés, la secuencia se recorre en sentido contrario: 00 → 01 → 11 → 10. Entonces, conociendo el estado anterior y el nuevo, sabés para qué lado se movió. Y si alguna vez cambian los dos bits a la vez (00 → 11), es un estado ilegal: se perdió un flanco o hubo ruido.

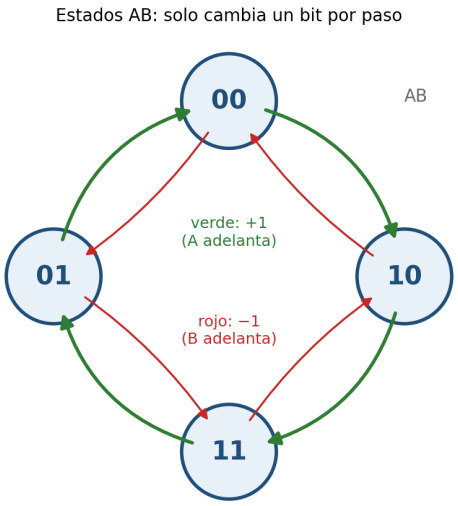


Figura 2 — Máquina de estados de la cuadratura. Cada flecha es un flanco; su sentido de recorrido es el sentido de giro.

PPR, CPR y los modos x1, x2, x4

Acá hay una confusión de nombres muy común, así que conviene fijarla:

- **PPR** (pulsos por revolución): cuántos **períodos completos** tiene la señal A en una vuelta. En el motor CQRobot: 16 pulsos por vuelta del eje del motor, por canal.
- **CPR** (cuentas por revolución): cuántas **cuentas** obtenés por vuelta. Depende del modo de decodificación. El fabricante declara 64 CPR, que son 16 PPR × 4: o sea, CPR en modo x4.
- **x1**: contás solo las subidas de A (1 cuenta por período). **x2**: subidas y bajadas de A. **x4**: subidas y bajadas de A y de B. Mismo encoder, cuatro veces más resolución, gratis.

Modo	Cuentas por vuelta del motor	Cuentas por vuelta de rueda (×90)	Resolución angular de rueda
x1	16	1440	0,25°
x2	32	2880	0,125°
x4 (el del rover)	64	5760	0,0625°

Los tiempos que hay que respetar

Con el motor a 120 rpm de salida, cada canal tiene 2880 Hz → período de ≈347 µs. Como los flancos de A y B están separados un cuarto de período, **el evento más corto que el sistema tiene que distinguir dura ≈87 µs** (la nota del rover redondea a 2,9 kHz y 86 µs; es lo mismo). Este número es la vara con la que se mide todo lo demás: la subida del pull-up, el filtro de glitch y cualquier RC tienen que ser mucho más cortos que 86 µs.

3. Cómo es el PCNT por dentro

La Figura 3 muestra una unidad PCNT. El ESP32 (el clásico, el del rover) tiene **8 unidades independientes**. Cada unidad tiene:

- Un **contador de 16 bits con signo** (de −32768 a 32767).
- **Dos canales** (canal 0 y canal 1). Cada canal tiene dos entradas: una entrada de **flanco** (*edge*, la señal que se cuenta) y una entrada de **nivel** (*level* o control, que modifica qué se hace con cada flanco). Los dos canales actúan sobre el **mismo** contador.
- Un **filtro de glitch** que descarta pulsos más cortos que un umbral configurable.
- **Comparadores** que generan eventos: límite alto (h_lim), límite bajo (l_lim), dos umbrales (thres0, thres1) y cruce por cero. Los eventos pueden disparar una interrupción. Al llegar a h_lim o l_lim, el contador **vuelve a 0**.
- Las entradas llegan por la **matriz GPIO**, así que cualquier pin de entrada puede ir a cualquier canal, incluidos IO34–IO39 (que son solo entrada).

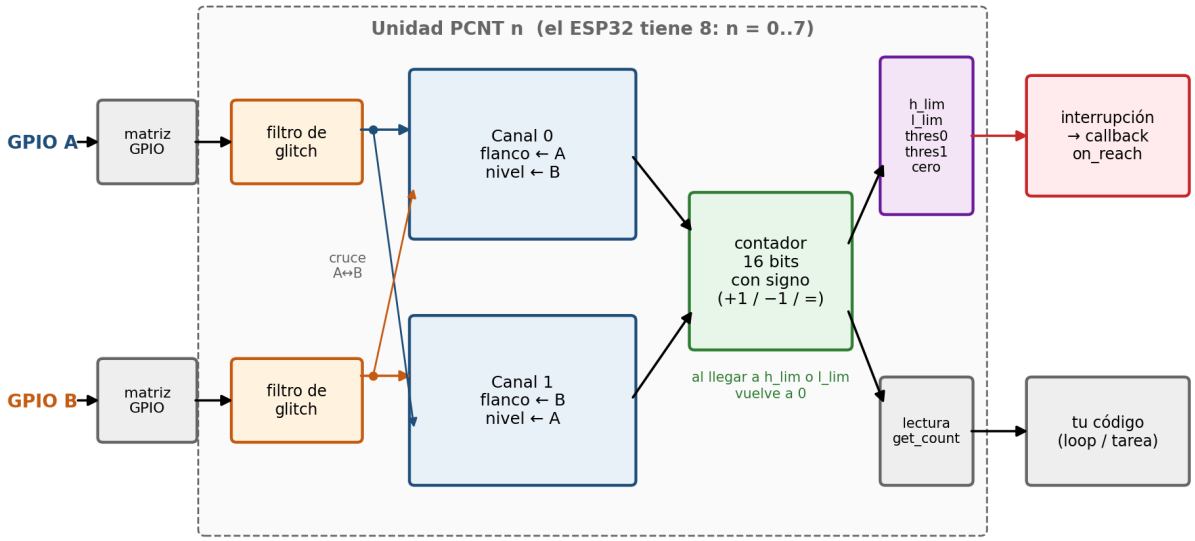


Figura 3 — Una unidad PCNT configurada como decodificador de cuadratura: cada canal toma el flanco de una señal y el nivel de la otra.

Las acciones: qué hace el contador en cada flanco

Toda la configuración se reduce a dos tablas por canal. La regla para leerlas: **el flanco dice qué hacer; el nivel dice si eso se respeta, se invierte o se congela**.

Acción de flanco (edge)	Efecto
`PCNT_CHANNEL_EDGE_ACTION_INCREASE`	Suma 1 al contador
`PCNT_CHANNEL_EDGE_ACTION_DECREASE`	Resta 1 al contador
`PCNT_CHANNEL_EDGE_ACTION_HOLD`	No hace nada con ese flanco

Acción de nivel (level)	Efecto sobre la acción de flanco
`PCNT_CHANNEL_LEVEL_ACTION_KEEP`	La respeta tal cual
`PCNT_CHANNEL_LEVEL_ACTION_INVERSE`	La invierte (sumar ↔ restar)
`PCNT_CHANNEL_LEVEL_ACTION_HOLD`	La anula: el contador queda congelado mientras dure ese nivel

Se configuran con dos funciones. **Ojo con el orden de los argumentos**, que es la fuente de error más común:

```
pcnt_channel_set_edge_action (canal, accion_en_SUBIDA, accion_en_BAJADA);
pcnt_channel_set_level_action(canal, accion_con_nivel_ALTO, accion_con_nivel_BAJO);
```

4. Decodificación x4 con dos canales

La configuración que usa esta guía es la del ejemplo oficial de Espressif para encoders rotativos:

	Entrada de flanco	Entrada de nivel	Subida	Bajada	Nivel alto	Nivel bajo
Canal 0	A	B	DECREASE	INCREASE	KEEP	INVERSE
Canal 1	B	A	INCREASE	DECREASE	KEEP	INVERSE

Verifiquemos que con A adelantando a B todas las transiciones dan +1. Para cada transición, uno solo de los dos canales ve un flanco (el de la señal que cambió); ese canal mira el nivel de la otra señal:

Transición AB	Cambió	Canal que actúa	Acción base	Nivel de la otra	Modificador	Resultado
00 → 10	A sube	Canal 0	DECREASE	B = 0	INVERSE	**+1**
10 → 11	B sube	Canal 1	INCREASE	A = 1	KEEP	**+1**
11 → 01	A baja	Canal 0	INCREASE	B = 1	KEEP	**+1**
01 → 00	B baja	Canal 1	DECREASE	A = 0	INVERSE	**+1**

En sentido contrario (00 → 01 → 11 → 10 → 00) cada flanco tiene el signo opuesto y el mismo nivel en la otra señal, así que todas las transiciones dan **−1**. Resultado: **cuatro cuentas por período, con signo**. Esto lo verifiqué simulando la lógica de las tablas de acciones para secuencias completas en ambos sentidos.

> **Para entenderlo.** ¿Por qué hacen falta dos canales? Un solo canal (flanco A, nivel B) ya da dirección, pero solo ve los flancos de A: sería x2. El segundo canal, con los roles invertidos, agrega los flancos de B. Por eso cada encoder consume una unidad completa y el rover usa 4 de las 8.

Dos propiedades que vas a ver en el banco

- **Un glitch simétrico se cancela solo.** Si A tiene un pulsito espurio (baja y vuelve a subir) con B quieto, el primer flanco suma y el segundo resta: neto 0. La cuadratura x4 es bastante tolerante al ruido; los errores reales aparecen cuando el ruido coincide con un flanco verdadero o cuando se pierde uno de los dos flancos del glitch.
- **Un canal muerto no da cero: da ±1.** Si B está desconectado o pegado, cada flanco de A suma y resta alternadamente: el contador oscila entre 0 y 1 mientras la rueda gira. Es un síntoma inconfundible (ver la sección 11).
- **Para invertir el sentido** alcanza con intercambiar A y B, o con multiplicar por −1 en software. En el rover conviene lo segundo: un signo por rueda en una tabla, sin tocar el cableado.

5. El filtro de glitch

Cada unidad tiene un filtro digital: una transición en la entrada solo se acepta si el nuevo nivel se mantiene estable durante al menos **N ciclos del reloj APB** (80 MHz → 12,5 ns por ciclo). Los pulsos más cortos se descartan como ruido. En el ESP32 el registro del filtro es de 10 bits: el máximo es **1023 ciclos = 12,79 µs**.

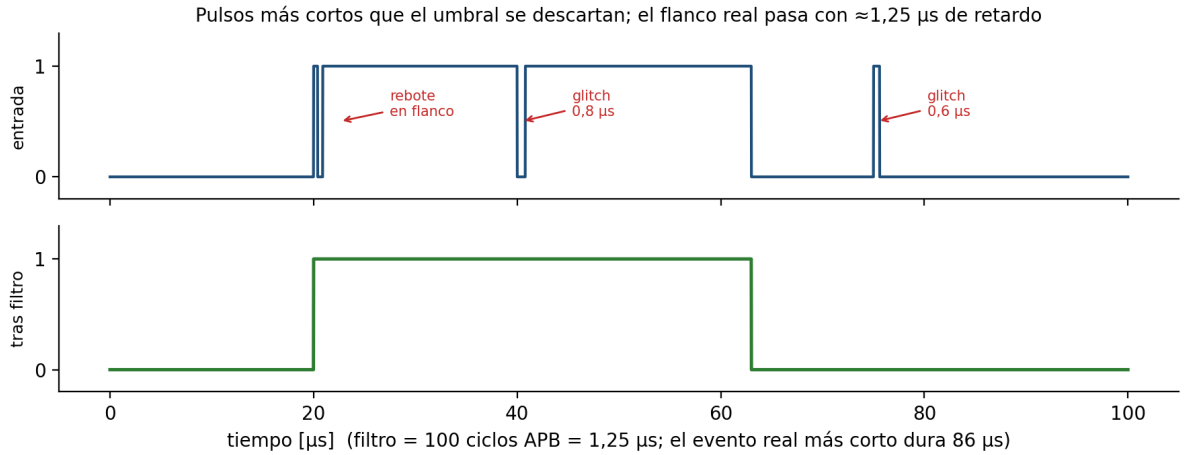


Figura 5 — El filtro descarta los pulsos de menos de 1,25 µs. Al flanco real solo le agrega un retardo fijo de ~1,25 µs, que no cambia la cuenta.

En el driver nuevo el umbral se da en nanosegundos con `max_glitch_ns`; el driver lo convierte a ciclos (ns × 80 / 1000). Si pedís más de ~12 787 ns, la función devuelve `ESP_ERR_INVALID_ARG`.

max_glitch_ns	Ciclos APB	Comparado con el evento de 86 µs	Uso
0 (filtro apagado)	—	—	Solo para pruebas
1250	100	1,5 %	Valor por defecto de esta guía
2500	200	2,9 %	Si aparecen cuentas espurias
12 787	1023 (máximo)	15 %	Techo del hardware

Criterio de diseño: el filtro tiene que ser mucho más corto que el evento real más corto. Con 86 µs de evento, cualquier valor de hasta unos pocos microsegundos es seguro, por eso hay tanto margen.

En el rover. La nota del rover define la secuencia de corrección si aparecen cuentas espurias con los motores girando: (1) activar o subir el filtro de glitch (gratis, sin componentes); (2) recién si no alcanza, bajar los pull-ups de 4,7 k a 2,2 k; (3) como último recurso, un RC de 2,2 k serie + 1 nF (τ = 2,2 µs). **No usar 10 nF**: τ = 22 µs es una cuarta parte del evento de 86 µs y deforma los flancos. Los ejemplos de esta guía dejan el filtro activo en 1250 ns porque no tiene costo; si querés reproducir exactamente la secuencia de la nota, poné `FILTRO_NS = 0` y activalo cuando aparezca ruido.

6. El desborde del contador de 16 bits

Este es **el** punto delicado del firmware, y la nota del rover lo marca como pendiente. A 120 rpm de salida la rueda genera 2 rev/s × 5760 = **11 520 cuentas/s**. El contador llega a 32 767 en 32 767 / 11 520 = **2,84 s**. Al llegar al límite, el hardware lo vuelve a 0 y sigue contando desde ahí.

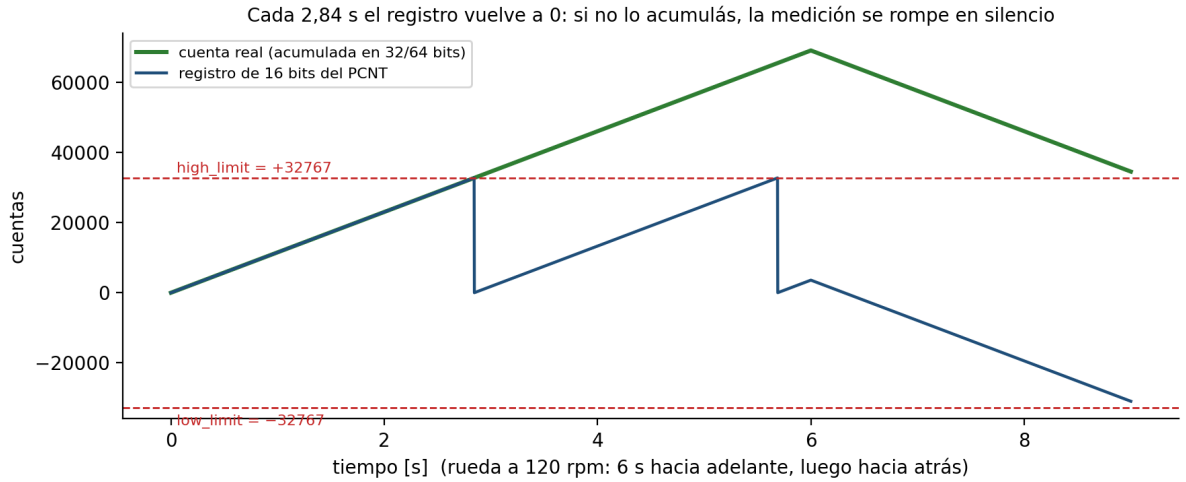


Figura 4 — Lo que ve el registro del PCNT (azul) frente a la posición real de la rueda (verde).

Si tu programa lee el registro y lo usa directamente, la posición «salta» de 32 766 a 0 y la velocidad calculada en ese instante es un pico enorme negativo. Nada falla ni avisa: la medición está mal y listo. Hay tres maneras de resolverlo:

Estrategia	Cómo funciona	Ventajas	Cuidado con
A. accum_count (driver)	Activás `flags.accum_count` y ponés watch points en los límites. El driver suma cada desborde y `pcnt_unit_get_count()` devuelve la posición acumulada (int de 32 bits).	Mínimo código. Ideal para posición.	Una interrupción cada 2,84 s por rueda (despreciable). El int de 32 bits desborda a las ~52 h de giro continuo.
B. Desenrollado por software	Leés el registro a período fijo y calculás la diferencia con la lectura anterior corrigiendo el salto del límite. Acumulás en int64.	Sin interrupciones. Sin condiciones de carrera. Te da posición y velocidad en el mismo paso.	Hay que leer al menos cada ~1,4 s. En un lazo de velocidad a 10 ms sobra.
C. Callback manual	Registrás un callback `on_reach` que suma el límite a un offset propio.	Sirve para entender qué hace A por dentro.	Condición de carrera: el contador vuelve a 0 en hardware unos µs antes de que corra el callback. Una lectura en ese hueco da un error de 32 767.

La matemática de la estrategia B

Con límites simétricos ±L (usamos L = 32 767), el registro siempre vale la posición real **módulo L**: cada vez que se reinicia, se «pierden» exactamente L cuentas. Si entre dos lecturas la rueda avanzó Δ cuentas reales, la diferencia de lecturas es Δ o Δ ± L. Como sabemos que |Δ| es chico, elegimos el representante en el intervalo (−L/2, L/2]:

$$d = c_k - c_{k-1} \;;\; \text{si } d > L/2 \rightarrow d = d - L \;;\; \text{si } d < -L/2 \rightarrow d = d + L$$

Ejemplo: lectura anterior 32 700, lectura actual 50. d = 50 − 32 700 = −32 650 < −16 383 → d = −32 650 + 32 767 = **+117** cuentas, que es exactamente lo que avanzó la rueda en 10 ms a 120 rpm (11 520 × 0,01 ≈ 115).

Condición de validez: |Δ real| < L/2 = 16 383 entre lecturas. A 11 520 cuentas/s eso es un período de lectura máximo de 1,42 s. Leyendo cada 10 ms hay un margen de más de 100 veces. Verifiqué el algoritmo con una simulación de 20 000 períodos aleatorios en ambos sentidos: la posición reconstruida coincide exactamente con la real.

Clave. Recomendación para el rover: **estrategia B** dentro de la tarea de control (Ejemplo 3), porque ya tenés que leer cada rueda a período fijo para calcular velocidad. Usá **A** cuando solo necesites posición con mínimo código (Ejemplo 2). En ambos casos se agregan watch points en ±L: dejan bien definido el reinicio a 0 en los límites, que es lo que suponen las dos estrategias.

7. El driver en Arduino IDE

Qué versión del core tenés (esto define todo)

Arduino IDE no tiene una API propia para el PCNT: se usa directamente el driver de ESP-IDF, que viene incluido en el core «esp32 by Espressif Systems». Según la versión del core, el driver es distinto:

Core Arduino-ESP32	ESP-IDF interno	Driver PCNT	Header
2.0.x	4.4	Viejo (legacy)	`driver/pcnt.h`
3.x (recomendado)	5.x	Nuevo (handles, el de esta guía)	`driver/pulse_cnt.h`

Para verla: **Herramientas → Placa → Gestor de placas**, buscá «esp32» y mirá la versión instalada de «esp32 by Espressif Systems». Si tenés la 2.0.x, actualizá a la 3.x. También podés comprobarlo en el programa:

Verificar la versión del core

```
#include "esp_arduino_version.h"
void setup() {
  Serial.begin(115200);
  delay(500);
  Serial.printf("Core Arduino-ESP32 %d.%d.%d\n", ESP_ARDUINO_VERSION_MAJOR,
                ESP_ARDUINO_VERSION_MINOR, ESP_ARDUINO_VERSION_PATCH);
}
void loop() {}
```

Atención. En ESP-IDF 5 **no se pueden mezclar** el driver viejo y el nuevo de un mismo periférico en el mismo firmware: el programa aborta al arrancar con un mensaje de conflicto. Esto incluye librerías de terceros: si alguna librería de encoders usa `driver/pcnt.h` y tu código usa `driver/pulse_cnt.h`, va a fallar. Elegí uno solo.

Ciclo de vida del driver

El driver nuevo trabaja con **handles** (punteros opacos) y tiene dos estados. Todo lo que es configuración se hace en estado INIT; después de `pcnt_unit_enable()` solo se puede arrancar, parar, leer y poner en cero.

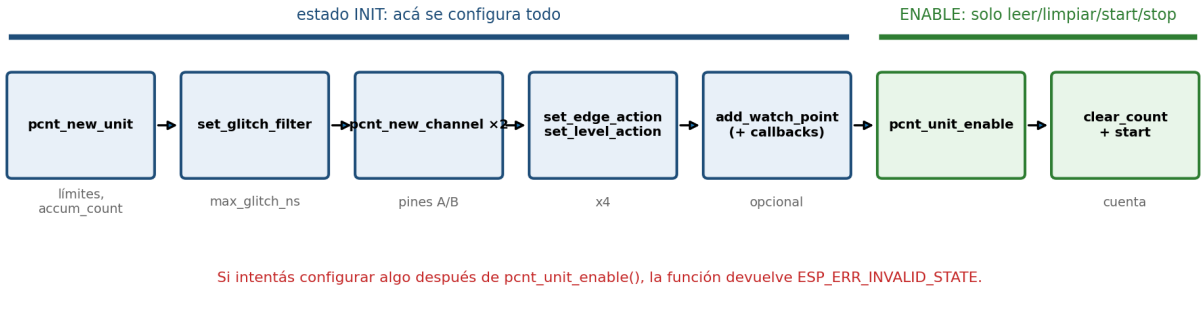


Figura 7 — Orden obligatorio de las llamadas.

Función	Qué hace	Estado
`pcnt_new_unit(&cfg;, &unidad;)`	Reserva una de las 8 unidades y fija los límites (y accum_count). El driver elige qué unidad física usar.	INIT
`pcnt_unit_set_glitch_filter(u, &f;)`	Configura el filtro. NULL lo desactiva.	INIT
`pcnt_new_channel(u, &cfg;, &canal;)`	Crea un canal y le asigna los GPIO de flanco y de nivel (−1 = IO virtual).	INIT
`pcnt_channel_set_edge_action(...)`	Acciones en subida y bajada.	cualquiera
`pcnt_channel_set_level_action(...)`	Acciones con nivel alto y bajo.	cualquiera
`pcnt_unit_add_watch_point(u, v)`	Genera un evento cuando el contador llega a v. Después hay que llamar a `clear_count` para que tome efecto.	INIT
`pcnt_unit_register_event_callbacks`	Registra el callback `on_reach` (corre en contexto de ISR).	INIT
`pcnt_unit_enable(u)`	Pasa a ENABLE; instala la interrupción si hay callbacks.	INIT → ENABLE
`pcnt_unit_clear_count(u)`	Pone el contador (y el acumulado) en 0.	ENABLE
`pcnt_unit_start(u) / _stop(u)`	Arranca o congela el conteo (stop no borra).	ENABLE
`pcnt_unit_get_count(u, &v;)`	Lee el valor (con signo). Se puede llamar desde una ISR.	ENABLE

Detalles de C++ en Arduino que conviene conocer

- **Inicializá las estructuras con `= {}` y asigná campo por campo.** El orden de los campos de `pcnt_unit_config_t` cambió entre versiones de ESP-IDF; los inicializadores designados de C++ exigen ese orden y pueden dejar de compilar al actualizar el core. Con `= {}` además todos los flags quedan en 0.
- `ESP_ERROR_CHECK(x)` detiene el programa e imprime el error si `x` no es `ESP_OK`. Está bien para pruebas; en el firmware final conviene revisar el valor y reportarlo (el Ejemplo 3 lo hace con `esp_err_to_name()`).
- **El driver no activa pull-ups ni pull-downs internos** en los pines del PCNT. En el rover no hace falta: la placa tiene pull-ups externos de 4,7 k, y IO34–IO39 ni siquiera tienen pull-up interno.
- **Cuántas unidades quedan:** si `pcnt_new_unit` devuelve `ESP_ERR_NOT_FOUND`, ya usaste las 8.

8. Ejemplos de código

Los seis ejemplos van de lo simple a lo completo. Todos compilan con el core 3.x; los pines son **de ejemplo** y hay que reemplazarlos por los de `40-firmware/mapa-gpio.md` antes de cargarlos en la placa del rover.

Ejemplo	Qué muestra	Cuándo usarlo
Ej0_Interrupciones	Decodificación x4 por software, con tabla de transiciones	Solo para comparar y entender la cuadratura
Ej1_PCNT_Minimo	Una unidad, dos canales, x4, lectura directa	Primer contacto con el driver
Ej2_PCNT_Acumulado	accum_count: posición de 32 bits sin desbordes	Posición con mínimo código
Ej3_CuatroRuedas	Clase EncoderPCNT, 4 ruedas, desenrollado, rpm, tarea FreeRTOS	Base del firmware del rover
Ej4_DiagnosticoCanales	8 unidades, una por cable, para validar A y B de cada rueda	Pendiente «probar los 4 canales»
Ej5_SimuladorLoopback	El ESP32 se genera su propia cuadratura y la cuenta	Probar el código sin motores

Ejemplo 0 — Interrupciones (para comparar)

Antes del PCNT, así se haría por software. Cada flanco de A o B dispara una ISR que lee ambos pines, arma el estado (A<<1)|B y consulta una tabla de 16 entradas indexada por (estado previo, estado nuevo). Es la Figura 2 convertida en tabla: las transiciones válidas dan ±1 y las ilegales (cambian los dos bits) dan 0.

Ej0_Interrupciones.ino

```
// Ej0_Interrupciones.ino
// Decodificacion x4 por interrupciones. SOLO para comparar con el PCNT:
// funciona, pero cada flanco interrumpe a la CPU.
#include <Arduino.h>

constexpr int PIN_A = 34;  // OJO: pines de ejemplo, completar desde 40-firmware/mapa-gpio.md
constexpr int PIN_B = 35;

volatile int32_t cuenta = 0;
volatile uint32_t interrupciones = 0;
volatile uint8_t estadoPrevio = 0;

// Tabla de transicion. Estado = (A << 1) | B. Indice = (previo << 2) | nuevo.
// +1: A adelanta a B | -1: B adelanta a A | 0: sin cambio o salto ilegal (2 bits a la vez)
DRAM_ATTR const int8_t TABLA[16] = {
   0, -1, +1,  0,     // previo = 00
  +1,  0,  0, -1,     // previo = 01
  -1,  0,  0, +1,     // previo = 10
   0, +1, -1,  0      // previo = 11
};

void IRAM_ATTR isrEncoder() {
  uint8_t nuevo = (digitalRead(PIN_A) << 1) | digitalRead(PIN_B);
  cuenta += TABLA[(estadoPrevio << 2) | nuevo];
  estadoPrevio = nuevo;
  interrupciones++;
}

void setup() {
  Serial.begin(115200);
  pinMode(PIN_A, INPUT);   // IO34..39 no tienen pull-up interno: usar el externo de la placa
  pinMode(PIN_B, INPUT);
  estadoPrevio = (digitalRead(PIN_A) << 1) | digitalRead(PIN_B);
  attachInterrupt(digitalPinToInterrupt(PIN_A), isrEncoder, CHANGE);
  attachInterrupt(digitalPinToInterrupt(PIN_B), isrEncoder, CHANGE);
}
```

```
void loop() {
  static uint32_t irqPrevias = 0;
  uint32_t irq = interrupciones;
  Serial.printf("cuenta = %ld   interrupciones/s = %lu\n",
                (long)cuenta, (unsigned long)(irq - irqPrevias));
  irqPrevias = irq;
  delay(1000);
}
```

Qué deberías ver: con el motor a pleno, unas 11 500 interrupciones por segundo para una sola rueda. Multiplicá por cuatro y tenés el argumento de la sección 1.

Ejemplo 1 — PCNT mínimo

Los cinco pasos de la Figura 7, uno detrás del otro. Fijate en los dos canales cruzados (paso 3) y en el orden de los argumentos de las acciones (paso 4).

Ej1_PCNT_Minimo.ino

```
// Ej1_PCNT_Minimo.ino
// Un encoder en cuadratura x4 con el driver nuevo de PCNT (core Arduino-ESP32 3.x).
#include <Arduino.h>
#include "esp_arduino_version.h"
#include "driver/pulse_cnt.h"

#if ESP_ARDUINO_VERSION_MAJOR < 3
#error "Este sketch usa driver/pulse_cnt.h: requiere el core Arduino-ESP32 3.x"
#endif

constexpr int PIN_A = 34;        // OJO: pines de ejemplo, completar desde mapa-gpio.md
constexpr int PIN_B = 35;
constexpr int LIMITE = 32767;    // maximo del contador de 16 bits con signo

pcnt_unit_handle_t unidad = nullptr;

void setup() {
  Serial.begin(115200);

  // 1) Unidad: rango del contador. El hardware vuelve a 0 al llegar a un limite.
  pcnt_unit_config_t cfgUnidad = {};
  cfgUnidad.high_limit = LIMITE;
  cfgUnidad.low_limit  = -LIMITE;
  ESP_ERROR_CHECK(pcnt_new_unit(&cfgUnidad, &unidad));

  // 2) Filtro de glitch: ignora pulsos de menos de 1250 ns (100 ciclos de APB a 80 MHz)
  pcnt_glitch_filter_config_t cfgFiltro = {};
  cfgFiltro.max_glitch_ns = 1250;
  ESP_ERROR_CHECK(pcnt_unit_set_glitch_filter(unidad, &cfgFiltro));

  // 3) Dos canales cruzados: canal A cuenta flancos de A mirando el nivel de B, y viceversa
  pcnt_chan_config_t cfgA = {};
  cfgA.edge_gpio_num  = PIN_A;
  cfgA.level_gpio_num = PIN_B;
  pcnt_channel_handle_t canalA = nullptr;
  ESP_ERROR_CHECK(pcnt_new_channel(unidad, &cfgA, &canalA));

  pcnt_chan_config_t cfgB = {};
  cfgB.edge_gpio_num  = PIN_B;
  cfgB.level_gpio_num = PIN_A;
  pcnt_channel_handle_t canalB = nullptr;
  ESP_ERROR_CHECK(pcnt_new_channel(unidad, &cfgB, &canalB));

  // 4) Acciones para x4. Orden de argumentos: (subida, bajada) y (nivel alto, nivel bajo)
  ESP_ERROR_CHECK(pcnt_channel_set_edge_action(canalA,
      PCNT_CHANNEL_EDGE_ACTION_DECREASE, PCNT_CHANNEL_EDGE_ACTION_INCREASE));
  ESP_ERROR_CHECK(pcnt_channel_set_level_action(canalA,
      PCNT_CHANNEL_LEVEL_ACTION_KEEP, PCNT_CHANNEL_LEVEL_ACTION_INVERSE));
  ESP_ERROR_CHECK(pcnt_channel_set_edge_action(canalB,
      PCNT_CHANNEL_EDGE_ACTION_INCREASE, PCNT_CHANNEL_EDGE_ACTION_DECREASE));
  ESP_ERROR_CHECK(pcnt_channel_set_level_action(canalB,
      PCNT_CHANNEL_LEVEL_ACTION_KEEP, PCNT_CHANNEL_LEVEL_ACTION_INVERSE));

  // 5) Habilitar, poner en cero y arrancar
```

```
  ESP_ERROR_CHECK(pcnt_unit_enable(unidad));
  ESP_ERROR_CHECK(pcnt_unit_clear_count(unidad));
  ESP_ERROR_CHECK(pcnt_unit_start(unidad));
}

void loop() {
  int cuenta = 0;
  pcnt_unit_get_count(unidad, &cuenta);
  // Ojo: con el motor a 120 rpm este valor vuelve a 0 cada ~2,8 s (ver Ej2 y Ej3)
  Serial.printf("cuenta = %6d   (%.3f vueltas de rueda)\n", cuenta, cuenta / 5760.0f);
  delay(100);
}
```

Qué deberías ver: girando la rueda a mano una vuelta, la cuenta cambia en ±5760. Con el motor a 120 rpm la cuenta sube hasta ~32 700 y vuelve a 0 cada 2,8 s: es el desborde de la sección 6, a propósito sin resolver.

Ejemplo 2 — Posición acumulada con accum_count

Tres cambios respecto del Ejemplo 1, marcados (1), (2) y (3) en los comentarios. Desde el programa no se nota ninguna diferencia salvo que la cuenta ya no vuelve a 0.

Ej2_PCNT_Acumulado.ino

```
// Ej2_PCNT_Acumulado.ino
// Igual que Ej1, pero el driver acumula los desbordes: la cuenta pasa a ser de 32 bits.
#include <Arduino.h>
#include "esp_arduino_version.h"
#include "driver/pulse_cnt.h"

#if ESP_ARDUINO_VERSION_MAJOR < 3
#error "Requiere el core Arduino-ESP32 3.x"
#endif

constexpr int PIN_A = 34;        // OJO: pines de ejemplo
constexpr int PIN_B = 35;
constexpr int LIMITE = 32767;
constexpr int CUENTAS_VUELTA = 64 * 90;   // 5760

pcnt_unit_handle_t unidad = nullptr;

void setup() {
  Serial.begin(115200);

  pcnt_unit_config_t cfgUnidad = {};
  cfgUnidad.high_limit = LIMITE;
  cfgUnidad.low_limit  = -LIMITE;
  cfgUnidad.flags.accum_count = 1;          // (1) el driver suma cada desborde
  ESP_ERROR_CHECK(pcnt_new_unit(&cfgUnidad, &unidad));

  pcnt_glitch_filter_config_t cfgFiltro = {};
  cfgFiltro.max_glitch_ns = 1250;
  ESP_ERROR_CHECK(pcnt_unit_set_glitch_filter(unidad, &cfgFiltro));

  pcnt_chan_config_t cfg = {};
  pcnt_channel_handle_t canalA = nullptr, canalB = nullptr;
  cfg.edge_gpio_num = PIN_A;  cfg.level_gpio_num = PIN_B;
  ESP_ERROR_CHECK(pcnt_new_channel(unidad, &cfg, &canalA));
  cfg.edge_gpio_num = PIN_B;  cfg.level_gpio_num = PIN_A;
  ESP_ERROR_CHECK(pcnt_new_channel(unidad, &cfg, &canalB));

  pcnt_channel_set_edge_action(canalA, PCNT_CHANNEL_EDGE_ACTION_DECREASE,
                               PCNT_CHANNEL_EDGE_ACTION_INCREASE);
  pcnt_channel_set_level_action(canalA, PCNT_CHANNEL_LEVEL_ACTION_KEEP,
                                PCNT_CHANNEL_LEVEL_ACTION_INVERSE);
  pcnt_channel_set_edge_action(canalB, PCNT_CHANNEL_EDGE_ACTION_INCREASE,
                               PCNT_CHANNEL_EDGE_ACTION_DECREASE);
  pcnt_channel_set_level_action(canalB, PCNT_CHANNEL_LEVEL_ACTION_KEEP,
                                PCNT_CHANNEL_LEVEL_ACTION_INVERSE);

  // (2) watch points en ambos limites: asi el driver se entera de cada desborde
  ESP_ERROR_CHECK(pcnt_unit_add_watch_point(unidad,  LIMITE));
```

```
  ESP_ERROR_CHECK(pcnt_unit_add_watch_point(unidad, -LIMITE));

  ESP_ERROR_CHECK(pcnt_unit_enable(unidad));
  ESP_ERROR_CHECK(pcnt_unit_clear_count(unidad));   // necesario tras agregar watch points
  ESP_ERROR_CHECK(pcnt_unit_start(unidad));
}

void loop() {
  int posicion = 0;
  pcnt_unit_get_count(unidad, &posicion);   // (3) ya incluye los desbordes acumulados
  Serial.printf("posicion = %ld cuentas = %.3f vueltas\n",
                (long)posicion, posicion / (float)CUENTAS_VUELTA);
  delay(200);
}
```

Ejemplo 3 — Las cuatro ruedas (base del firmware)

Este es el que conviene estudiar en detalle. Decisiones de diseño:

- **Una clase por encoder** (`EncoderPCNT`) que encapsula la unidad, el desenrollado y el signo de la rueda.
- **Tabla** `RUEDAS[]` con nombre, pines y signo: todo lo específico del cableado está en un solo lugar.
- **Una tarea FreeRTOS a período fijo** con `vTaskDelayUntil()`: a diferencia de `delay()`, compensa el tiempo de ejecución y no acumula deriva. El período constante es lo que hace que «cuentas por período» sea una velocidad.
- **Estrategia B de desenrollado** (sección 6): sin interrupciones, posición en 64 bits.
- **Sección crítica** (`portENTER_CRITICAL`) para copiar el resultado: un `int64_t` no se escribe de forma atómica en un procesador de 32 bits, y sin protección `loop()` podría leer una posición a medio actualizar.

Ej3_CuatroRuedas.ino

```
// Ej3_CuatroRuedas.ino
// 4 encoders x4 (una unidad PCNT por rueda), posicion de 64 bits y velocidad en rpm.
// La lectura se hace en una tarea FreeRTOS a periodo fijo; loop() solo imprime.
#include <Arduino.h>
#include "esp_arduino_version.h"
#include "driver/pulse_cnt.h"

#if ESP_ARDUINO_VERSION_MAJOR < 3
#error "Requiere el core Arduino-ESP32 3.x"
#endif

// ------------------------- Parametros del rover -------------------------
constexpr int      CPR_MOTOR      = 64;      // cuentas x4 por vuelta del eje del motor
constexpr int      REDUCCION      = 90;
constexpr int      CUENTAS_VUELTA = CPR_MOTOR * REDUCCION;   // 5760 por vuelta de rueda
constexpr int      PERIODO_MS     = 10;      // periodo de muestreo de velocidad
constexpr int      LIMITE         = 32767;   // limite simetrico del contador de 16 bits
constexpr uint32_t FILTRO_NS      = 1250;    // 100 ciclos de APB; 0 = sin filtro
constexpr float    ALFA_EMA       = 0.2f;    // suavizado de la velocidad (1 = sin filtro)

struct ConfigRueda { const char* nombre; int pinA; int pinB; int8_t signo; };

// OJO: PINES DE EJEMPLO. Reemplazar por los de 40-firmware/mapa-gpio.md.
// OJO: SIGNOS A DEFINIR EN BANCO: +1 si la cuenta crece cuando esa rueda empuja
//      al rover hacia adelante. Las ruedas de un lado giran al reves que las del otro.
const ConfigRueda RUEDAS[4] = {
  {"FR", 34, 35, +1},   // J9
  {"FL", 36, 39, -1},   // J16
  {"RL", 32, 33, -1},   // J18
  {"RR", 25, 26, +1},   // J8
};

static bool ok(esp_err_t e, const char* que) {
  if (e != ESP_OK) Serial.printf("  [ERROR] %s: %s\n", que, esp_err_to_name(e));
  return e == ESP_OK;
```

```
}

// ============================ Clase EncoderPCNT ============================
class EncoderPCNT {
public:
  bool begin(int pinA, int pinB, int8_t signo) {
    signo_ = signo;

    pcnt_unit_config_t cu = {};
    cu.high_limit = LIMITE;
    cu.low_limit  = -LIMITE;
    if (!ok(pcnt_new_unit(&cu, &unidad_), "pcnt_new_unit")) return false;

    if (FILTRO_NS > 0) {
      pcnt_glitch_filter_config_t cf = {};
      cf.max_glitch_ns = FILTRO_NS;
      if (!ok(pcnt_unit_set_glitch_filter(unidad_, &cf), "glitch_filter")) return false;
    }

    pcnt_channel_handle_t chA = nullptr, chB = nullptr;
    pcnt_chan_config_t cc = {};
    cc.edge_gpio_num = pinA;  cc.level_gpio_num = pinB;
    if (!ok(pcnt_new_channel(unidad_, &cc, &chA), "canal A")) return false;
    cc.edge_gpio_num = pinB;  cc.level_gpio_num = pinA;
    if (!ok(pcnt_new_channel(unidad_, &cc, &chB), "canal B")) return false;

    pcnt_channel_set_edge_action(chA, PCNT_CHANNEL_EDGE_ACTION_DECREASE,
                                      PCNT_CHANNEL_EDGE_ACTION_INCREASE);
    pcnt_channel_set_level_action(chA, PCNT_CHANNEL_LEVEL_ACTION_KEEP,
                                       PCNT_CHANNEL_LEVEL_ACTION_INVERSE);
    pcnt_channel_set_edge_action(chB, PCNT_CHANNEL_EDGE_ACTION_INCREASE,
                                      PCNT_CHANNEL_EDGE_ACTION_DECREASE);
    pcnt_channel_set_level_action(chB, PCNT_CHANNEL_LEVEL_ACTION_KEEP,
                                       PCNT_CHANNEL_LEVEL_ACTION_INVERSE);

    // Watch points en +-LIMITE: dejan bien definido el reinicio a 0 en los limites
    // (lo mismo que exige accum_count). Sin callback registrado no hay interrupciones.
    if (!ok(pcnt_unit_add_watch_point(unidad_,  LIMITE), "watch +")) return false;
    if (!ok(pcnt_unit_add_watch_point(unidad_, -LIMITE), "watch -")) return false;

    if (!ok(pcnt_unit_enable(unidad_), "enable")) return false;
    pcnt_unit_clear_count(unidad_);
    pcnt_unit_start(unidad_);
    ultimo_ = 0;
    pos_ = 0;
    return true;
  }

  // Llamar a periodo fijo. Devuelve las cuentas avanzadas desde la llamada anterior,
  // ya con el signo de la rueda, y acumula la posicion en 64 bits.
  int32_t actualizar() {
    int ahora = 0;
    pcnt_unit_get_count(unidad_, &ahora);
    int32_t d = ahora - ultimo_;
    ultimo_ = ahora;
    // El registro es periodico con periodo LIMITE: llevamos d al rango (-LIMITE/2, LIMITE/2]
    while (d >  LIMITE / 2) d -= LIMITE;
    while (d < -LIMITE / 2) d += LIMITE;
    d *= signo_;
    pos_ += d;
    return d;
  }

  int64_t posicion() const { return pos_; }

private:
  pcnt_unit_handle_t unidad_ = nullptr;
  int     ultimo_ = 0;
  int64_t pos_    = 0;
  int8_t  signo_  = 1;
};

// ======================= Estado compartido y tarea =======================
EncoderPCNT enc[4];

struct Medicion { int64_t pos[4]; float rpm[4]; };
```

```
Medicion compartida = {};                       // la escribe la tarea, la lee loop()
portMUX_TYPE mux = portMUX_INITIALIZER_UNLOCKED;

void tareaEncoders(void*) {
  const float K_RPM = 60000.0f / (CUENTAS_VUELTA * PERIODO_MS);   // rpm por cuenta
  float rpmFiltrada[4] = {0, 0, 0, 0};
  TickType_t t0 = xTaskGetTickCount();
  for (;;) {
    vTaskDelayUntil(&t0, pdMS_TO_TICKS(PERIODO_MS));   // periodo fijo, sin deriva
    Medicion m;
    for (int i = 0; i < 4; i++) {
      int32_t d = enc[i].actualizar();
      rpmFiltrada[i] += ALFA_EMA * (d * K_RPM - rpmFiltrada[i]);
      m.pos[i] = enc[i].posicion();
      m.rpm[i] = rpmFiltrada[i];
    }
    portENTER_CRITICAL(&mux);
    compartida = m;
    portEXIT_CRITICAL(&mux);
  }
}

void setup() {
  Serial.begin(115200);
  delay(300);
  for (int i = 0; i < 4; i++) {
    Serial.printf("Rueda %s: A=IO%d B=IO%d\n", RUEDAS[i].nombre, RUEDAS[i].pinA,
                  RUEDAS[i].pinB);
    if (!enc[i].begin(RUEDAS[i].pinA, RUEDAS[i].pinB, RUEDAS[i].signo)) {
      Serial.println("Fallo la configuracion. Programa detenido.");
      while (true) delay(1000);
    }
  }
  xTaskCreatePinnedToCore(tareaEncoders, "encoders", 4096, nullptr, 5, nullptr, 1);
}

void loop() {
  Medicion m;
  portENTER_CRITICAL(&mux);
  m = compartida;
  portEXIT_CRITICAL(&mux);
  for (int i = 0; i < 4; i++) {
    Serial.printf("%s %9lld (%7.2f v) %7.1f rpm | ", RUEDAS[i].nombre,
                  (long long)m.pos[i], m.pos[i] / (float)CUENTAS_VUELTA, m.rpm[i]);
  }
  Serial.println();
  delay(200);
}
```

Qué deberías ver: una línea cada 200 ms con posición (cuentas y vueltas) y velocidad de cada rueda. Con las cuatro ruedas empujando hacia adelante, todas las posiciones tienen que crecer; si alguna decrece, cambiá su signo en la tabla.

Ejemplo 4 — Diagnóstico de los 8 cables

Truco: como el rover usa 4 de las 8 unidades, para diagnosticar se pueden usar las 8, **una por cable**. Cada unidad cuenta subidas y bajadas de una sola señal, sin señal de control (se usa un **IO virtual** fijo en alto: `level_gpio_num = -1`). Así ves por separado si A y B de cada rueda están vivos, cuántos flancos dan y en qué nivel están ahora.

Ej4_DiagnosticoCanales.ino

```
// Ej4_DiagnosticoCanales.ino
// Valida los 8 cables (A y B de las 4 ruedas) por separado: una unidad PCNT por cable,
// contando subida Y bajada. Sirve para el pendiente "repetir la prueba en los 4 canales".
// Procedimiento: enviar 'r' por el monitor serie, girar cada rueda UNA vuelta a mano.
// Esperado por cable: 16 pulsos x 2 flancos x 90 = 2880 flancos por vuelta de rueda.
#include <Arduino.h>
#include "esp_arduino_version.h"
#include "driver/pulse_cnt.h"
#include "driver/gpio.h"
```

```
#if ESP_ARDUINO_VERSION_MAJOR < 3
#error "Requiere el core Arduino-ESP32 3.x"
#endif

constexpr int LIMITE = 32767;
// OJO: pines de ejemplo, en el mismo orden que mapa-gpio.md
const int   PINES[8]   = { 34,     35,     36,     39,     32,     33,     25,     26   };
const char* NOMBRES[8] = {"FR.A", "FR.B", "FL.A", "FL.B", "RL.A", "RL.B", "RR.A", "RR.B"};

pcnt_unit_handle_t unidad[8];

void setup() {
  Serial.begin(115200);
  delay(300);
  for (int i = 0; i < 8; i++) {
    pcnt_unit_config_t cu = {};
    cu.high_limit = LIMITE;
    cu.low_limit  = -LIMITE;
    ESP_ERROR_CHECK(pcnt_new_unit(&cu, &unidad[i]));

    pcnt_glitch_filter_config_t cf = {};
    cf.max_glitch_ns = 1250;
    ESP_ERROR_CHECK(pcnt_unit_set_glitch_filter(unidad[i], &cf));

    pcnt_chan_config_t cc = {};
    cc.edge_gpio_num  = PINES[i];
    cc.level_gpio_num = -1;              // sin senial de control: IO virtual...
    cc.flags.virt_level_io_level = 1;    // ...fija en alto
    pcnt_channel_handle_t ch = nullptr;
    ESP_ERROR_CHECK(pcnt_new_channel(unidad[i], &cc, &ch));
    // Ambos flancos suman: medimos actividad del cable, no sentido de giro
    ESP_ERROR_CHECK(pcnt_channel_set_edge_action(ch, PCNT_CHANNEL_EDGE_ACTION_INCREASE,
                                                     PCNT_CHANNEL_EDGE_ACTION_INCREASE));
    ESP_ERROR_CHECK(pcnt_unit_enable(unidad[i]));
    ESP_ERROR_CHECK(pcnt_unit_clear_count(unidad[i]));
    ESP_ERROR_CHECK(pcnt_unit_start(unidad[i]));
  }
  Serial.println("Listo. 'r' = poner contadores en cero.");
}

void loop() {
  if (Serial.available() && Serial.read() == 'r') {
    for (int i = 0; i < 8; i++) pcnt_unit_clear_count(unidad[i]);
  }
  for (int i = 0; i < 8; i += 2) {         // de a pares: A y B de la misma rueda
    int fa = 0, fb = 0;
    pcnt_unit_get_count(unidad[i], &fa);
    pcnt_unit_get_count(unidad[i + 1], &fb);
    int na = gpio_get_level((gpio_num_t)PINES[i]);
    int nb = gpio_get_level((gpio_num_t)PINES[i + 1]);
    Serial.printf("%s %5d [%d]  %s %5d [%d]  dif %3d || ", NOMBRES[i], fa, na,
                  NOMBRES[i + 1], fb, nb, fa - fb);
  }
  Serial.println();
  delay(500);
}
```

Qué deberías ver: tras enviar 'r' y girar una rueda exactamente una vuelta, sus dos cables marcan ≈2880 flancos cada uno y la diferencia A−B es de 0 a ±2. Un cable con 0 flancos está cortado, sin alimentación o mal asignado. El nivel entre corchetes debe cambiar entre 0 y 1 al mover la rueda despacio; si queda fijo en 1 con la rueda girando, la línea no está llegando (el pull-up la sostiene alta).

Ejemplo 5 — Simulador por loopback

Para probar el código sin motores ni encoders. Dos salidas generan la secuencia 10 → 11 → 01 → 00 (A adelantando) con cuartos de período de 86 µs, como el CQRobot a pleno, y se puentean con cables a dos entradas que cuenta el PCNT. Ida: tiene que dar exactamente 4 × CICLOS. Ida y vuelta: exactamente 0. Ideal en un DevKit suelto.

Ej5_SimuladorLoopback.ino

```
// Ej5_SimuladorLoopback.ino
// Prueba del codigo SIN motor: el mismo ESP32 genera una cuadratura por dos salidas
// y la cuenta con PCNT por dos entradas. Puentear OUT_A->IN_A y OUT_B->IN_B con cables.
// Ideal: hacerlo en un DevKit suelto. No usar pines de encoders ni de arranque.
#include <Arduino.h>
#include "esp_arduino_version.h"
#include "driver/pulse_cnt.h"

#if ESP_ARDUINO_VERSION_MAJOR < 3
#error "Requiere el core Arduino-ESP32 3.x"
#endif

constexpr int OUT_A = 16, OUT_B = 17;      // salidas del "encoder falso" (ejemplo)
constexpr int IN_A  = 27, IN_B  = 14;      // entradas del PCNT           (ejemplo)
constexpr int CUARTO_US = 86;              // 1/4 de periodo: 2,9 kHz como el CQRobot
constexpr int CICLOS = 2000;               // ciclos completos por prueba
constexpr int LIMITE = 32767;

pcnt_unit_handle_t unidad = nullptr;

void generar(int ciclos, bool adelante) {
  // Secuencia AB para A adelantando a B: 10 -> 11 -> 01 -> 00
  static const uint8_t SEC[4][2] = {{1, 0}, {1, 1}, {0, 1}, {0, 0}};
  for (int c = 0; c < ciclos; c++) {
    for (int k = 0; k < 4; k++) {
      int j = adelante ? k : (6 - k) % 4;   // atras: 01 -> 11 -> 10 -> 00
      digitalWrite(OUT_A, SEC[j][0]);
      digitalWrite(OUT_B, SEC[j][1]);
      delayMicroseconds(CUARTO_US);
    }
  }
}

int leer() { int c = 0; pcnt_unit_get_count(unidad, &c); return c; }

void setup() {
  Serial.begin(115200);
  delay(300);
  pinMode(OUT_A, OUTPUT); digitalWrite(OUT_A, LOW);
  pinMode(OUT_B, OUTPUT); digitalWrite(OUT_B, LOW);

  pcnt_unit_config_t cu = {};
  cu.high_limit = LIMITE;
  cu.low_limit  = -LIMITE;
  ESP_ERROR_CHECK(pcnt_new_unit(&cu, &unidad));
  pcnt_glitch_filter_config_t cf = {};
  cf.max_glitch_ns = 1250;
  ESP_ERROR_CHECK(pcnt_unit_set_glitch_filter(unidad, &cf));

  pcnt_chan_config_t cc = {};
  pcnt_channel_handle_t chA = nullptr, chB = nullptr;
  cc.edge_gpio_num = IN_A;  cc.level_gpio_num = IN_B;
  ESP_ERROR_CHECK(pcnt_new_channel(unidad, &cc, &chA));
  cc.edge_gpio_num = IN_B;  cc.level_gpio_num = IN_A;
  ESP_ERROR_CHECK(pcnt_new_channel(unidad, &cc, &chB));
  pcnt_channel_set_edge_action(chA, PCNT_CHANNEL_EDGE_ACTION_DECREASE,
                                    PCNT_CHANNEL_EDGE_ACTION_INCREASE);
  pcnt_channel_set_level_action(chA, PCNT_CHANNEL_LEVEL_ACTION_KEEP,
                                     PCNT_CHANNEL_LEVEL_ACTION_INVERSE);
  pcnt_channel_set_edge_action(chB, PCNT_CHANNEL_EDGE_ACTION_INCREASE,
                                    PCNT_CHANNEL_EDGE_ACTION_DECREASE);
  pcnt_channel_set_level_action(chB, PCNT_CHANNEL_LEVEL_ACTION_KEEP,
                                     PCNT_CHANNEL_LEVEL_ACTION_INVERSE);
  ESP_ERROR_CHECK(pcnt_unit_enable(unidad));
  ESP_ERROR_CHECK(pcnt_unit_clear_count(unidad));
  ESP_ERROR_CHECK(pcnt_unit_start(unidad));
}

void loop() {
  pcnt_unit_clear_count(unidad);
  generar(CICLOS, true);
  int ida = leer();
  generar(CICLOS, false);
  int vuelta = leer();
  bool pasa = (ida == 4 * CICLOS) && (vuelta == 0);
```

```
  Serial.printf("adelante: %d (esperado %d)  ida y vuelta: %d (esperado 0)  -> %s\n",
                ida, 4 * CICLOS, vuelta, pasa ? "PASA" : "FALLA");
  delay(2000);
}
```

9. De cuentas a velocidad

Si en un período de muestreo T la rueda avanzó Δn cuentas, y una vuelta son N = 5760 cuentas:

$$\omega\ [\text{rpm}] = (\Delta n / N) \cdot (60 / T) \rightarrow K = 60\,000 / (N \cdot T_{ms})\ \text{rpm por cuenta}$$

Con T = 10 ms: K = 60 000 / (5760 × 10) = **1,042 rpm por cuenta**. Esa es la resolución de la medición: no podés distinguir 60,0 de 60,5 rpm en una sola muestra. El compromiso está en T:

T	Resolución (rpm/cuenta)	Cuentas por período a 120 rpm	Error de cuantización a 120 rpm	A 10 rpm
5 ms	2,083	57,6	1,7 %	21 %
10 ms	1,042	115,2	0,9 %	10 %
20 ms	0,521	230,4	0,4 %	5 %
50 ms	0,208	576	0,2 %	2 %

Período corto = respuesta rápida pero cuantización gruesa, sobre todo a baja velocidad. 10 ms es un buen punto de partida para el lazo de velocidad del modelo a escala. Para suavizar, el Ejemplo 3 usa un filtro exponencial (EMA):

$$\omega_f \leftarrow \omega_f + \alpha \cdot (\omega_{medida} - \omega_f)\ \text{constante de tiempo} \approx T \cdot (1 - \alpha) / \alpha$$

Con α = 0,2 y T = 10 ms, la constante de tiempo es ≈40 ms: suaviza la cuantización sin agregar un retardo que moleste a un PID de velocidad. Si a muy baja velocidad necesitás más resolución, las opciones son promediar varias muestras (ventana más larga) o medir el tiempo entre flancos, que ya no es tarea del PCNT.

Para pasar a velocidad lineal: v = ω · π · D / 60, con D el diámetro de rueda del modelo a escala (pendiente de revisar con Mecánica según la nota de diseño).

10. Hardware: qué hace la placa por el firmware

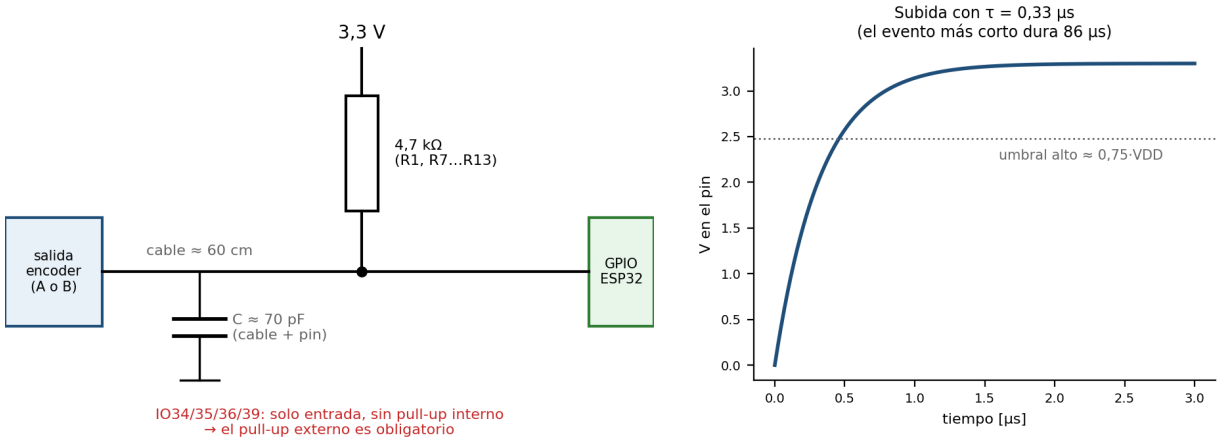


Figura 6 — Acondicionamiento de cada canal en la placa del modelo a escala y la subida resultante en el pin.

- **Pull-up de 4,7 k a 3,3 V obligatorio** en cada canal (R1, R7…R13). IO34, IO35, IO36 e IO39 son solo entrada y no tienen pull-up interno: sin la resistencia externa, la línea flota y el PCNT cuenta ruido.

- **Subida:** $\tau = 4{,}7\ \text{k} \times 70\ \text{pF} \approx 0{,}33\ \mu\text{s}$, tres órdenes de magnitud por debajo de los 86 µs del evento más corto. El valor está elegido por velocidad; lo que se resigna es algo de inmunidad al ruido (de ahí la secuencia de corrección de la sección 5).
- **Encoders a 3,3 V:** el encoder del CQRobot acepta 3,3–24 V. Alimentándolo con 3,3 V su salida no puede superar 3,3 V, lo que elimina de raíz el riesgo de sobretensión en IO34–IO39. Si un módulo trae su propio pull-up a VCC queda en paralelo ($4{,}7\ \text{k} \parallel 10\ \text{k} \approx 3{,}2\ \text{k}$): no es un problema, solo una pequeña diferencia de tiempos entre canales.
- **Cableado:** A y B como par trenzado con su propio GND de retorno, lejos de los cables de motor (pendiente de la nota).

11. Diagnóstico de problemas

Síntoma	Causa probable	Qué hacer
No compila: `driver/pulse_cnt.h: No such file`	Core 2.0.x	Actualizar a core 3.x (sección 7)
Aborta al arrancar con un mensaje de «CONFLICT»	Mezcla del driver viejo y el nuevo (a veces vía una librería)	Usar un solo driver en todo el firmware
La cuenta queda siempre en 0	Pin equivocado, falta `pcnt_unit_start()`, encoder sin alimentación	Revisar mapa-gpio.md; medir 3,3 V en el conector; correr Ej4
Oscila entre 0 y 1 mientras la rueda gira	Un canal (A o B) muerto o pegado	Ej4: el cable con 0 flancos es el culpable
Una vuelta da 2880 en vez de 5760	Decodificando en x2: falta el segundo canal o tiene acciones HOLD	Revisar que ambos canales estén creados y cruzados
Una vuelta da −5760	Sentido opuesto al definido	Cambiar el signo de esa rueda en RUEDAS[]
Saltos de ≈32 767 en posición, picos en velocidad	Desborde no manejado	Usar Ej2 (accum_count) o Ej3 (desenrollado)
La cuenta deriva lentamente con el motor quieto y PWM activo	Ruido acoplado desde los cables de motor	Filtro de glitch → pull-up 2,2 k → RC 2,2 k + 1 nF; separar y trenzar cables
Cambia en ±1 con la rueda quieta	La rueda vibra justo sobre un flanco	Normal en x4; no acumula error
`ESP_ERR_INVALID_STATE`	Configurar después de enable, o watch point repetido	Respetar el orden de la Figura 7
`ESP_ERR_NOT_FOUND` en new_unit	Ya hay 8 unidades en uso	No combinar Ej4 con otro código que use PCNT
`ESP_ERR_INVALID_ARG` en el filtro	`max_glitch_ns` mayor a ~12 787	Bajar el valor

12. Checklist de banco para el rover

Ordenado para cerrar los pendientes de la nota «Encoders y PCNT», de lo que no necesita hardware a lo que necesita todo:

1. **Código sin hardware** — Ej5 en un DevKit suelto: «PASA» en varias corridas.
2. **Los 8 cables** — Ej4 en la placa: cada cable ≈2880 flancos por vuelta de rueda, A−B entre 0 y ±2, niveles que cambian. Cierra el pendiente «repetir la prueba en los cuatro canales».
3. **Signo de cada rueda** — Ej3: empujar el rover hacia adelante una vuelta de rueda; cada posición debe aumentar ≈5760. Corregir la tabla RUEDAS[] y dejar documentado el sentido positivo de cada rueda.

4. **Velocidad a pleno** — motores sin carga: ≈120 rpm en las cuatro ruedas, sin saltos durante más de 3 s (verifica el manejo del desborde).
5. **Ruido** — motores con PWM y ruedas frenadas o quietas: la posición no debe derivar. Si deriva, aplicar la secuencia de corrección de la sección 5.
6. **Consumo** — medir la corriente de los cuatro encoders a 3,3 V (entra en el presupuesto del AMS1117).
7. **Cableado definitivo** — A/B en par trenzado con GND de retorno propio, lejos de los cables de motor; repetir el punto 5.

Apéndice A — El driver viejo (core 2.0.x)

Por si encontrás código o librerías anteriores. La lógica es idéntica (dos canales cruzados), cambian los nombres: no hay handles, se elige a mano la unidad (`PCNT_UNIT_0`…`PCNT_UNIT_7`), el filtro se da en ciclos de APB y la lectura es un `int16_t`. No mezclar con el driver nuevo.

Equivalente del Ejemplo 1 con el driver viejo

```
#include "driver/pcnt.h"     // SOLO core 2.0.x (ESP-IDF 4.4)

void configurarEncoderLegacy(pcnt_unit_t unidad, int pinA, int pinB) {
  pcnt_config_t c = {};
  c.unit = unidad;
  c.counter_h_lim =  32767;
  c.counter_l_lim = -32767;
  // Canal 0: flanco A, control B
  c.channel = PCNT_CHANNEL_0;  c.pulse_gpio_num = pinA;  c.ctrl_gpio_num = pinB;
  c.pos_mode = PCNT_COUNT_DEC;     c.neg_mode = PCNT_COUNT_INC;
  c.hctrl_mode = PCNT_MODE_KEEP;   c.lctrl_mode = PCNT_MODE_REVERSE;
  pcnt_unit_config(&c);
  // Canal 1: flanco B, control A
  c.channel = PCNT_CHANNEL_1;  c.pulse_gpio_num = pinB;  c.ctrl_gpio_num = pinA;
  c.pos_mode = PCNT_COUNT_INC;     c.neg_mode = PCNT_COUNT_DEC;
  pcnt_unit_config(&c);

  pcnt_set_filter_value(unidad, 100);   // en ciclos de APB (max 1023)
  pcnt_filter_enable(unidad);
  pcnt_counter_pause(unidad);
  pcnt_counter_clear(unidad);
  pcnt_counter_resume(unidad);
}

int16_t leerLegacy(pcnt_unit_t unidad) {
  int16_t v = 0;
  pcnt_get_counter_value(unidad, &v);
  return v;
}
```

Sobre la librería **ESP32Encoder** (muy usada en Arduino): envuelve el PCNT en una clase cómoda. Es una alternativa válida, pero verificá qué driver usa la versión que instales para no generar el conflicto de la sección 7. Para el rover, escribir la clase propia (Ejemplo 3) tiene la ventaja de que entendés y controlás cada línea.

Apéndice B — Números del rover en una tabla

Magnitud	Valor	De dónde sale
Pulsos por canal por vuelta del motor (PPR)	16	64 CPR / 4
Cuentas x4 por vuelta de rueda	5760	64 × 90
Resolución angular de rueda	0,0625°	360° / 5760
Frecuencia por canal a 120 rpm	2880 Hz	180 rev/s × 16
Período / evento más corto	≈347 µs / ≈87 µs	1/2880 y un cuarto de eso
Cuentas por segundo a 120 rpm	11 520	2 rev/s × 5760
Tiempo hasta desbordar 16 bits	2,84 s	32 767 / 11 520
Máximo período de lectura (estrategia B)	1,42 s	16 383 / 11 520
Desborde de un int32 acumulado	≈52 h	2^{31} / 11 520 s
τ de subida del pull-up	0,33 µs	4,7 kΩ × 70 pF
Filtro de glitch por defecto	1250 ns = 100 ciclos	1,5 % del evento de 86 µs
Máximo filtro del hardware	12,79 µs = 1023 ciclos	registro de 10 bits
Resolución de velocidad con T = 10 ms	1,042 rpm/cuenta	60 000 / (5760 × 10)
Unidades PCNT usadas / disponibles	4 / 8	una por rueda

Referencia principal: ESP-IDF Programming Guide, «Pulse Counter (PCNT)», driver `driver/pulse_cnt.h`*, y el ejemplo oficial* `peripherals/pcnt/rotary_encoder`*.*